

PUC-SP

Atividade: Inteligência Artificial - Machine Learning

Curso: Ciência da Computação

Equipe: Luan Bonasorte Capella e Matheus Heimrath Barbosa

Classificação Geográfica com Random Forest e Análise Exploratória de Dados (EDA)

Nesta atividade, exploramos o algoritmo **Random Forest** da biblioteca `scikit-learn` utilizando o conjunto de dados **Airports** da biblioteca `vega_datasets`.

O objetivo é prever a localização (estado) de aeroportos baseando-se em suas coordenadas geográficas (latitude e longitude). O projeto está dividido em duas abordagens de modelagem:

1. Avaliação completa com todos os estados americanos presentes no dataset.
2. Avaliação focada nos 5 estados com o maior volume de aeroportos, demonstrando o comportamento do algoritmo diante de fronteiras geográficas isoladas e contíguas.

```
# Importando as bibliotecas essenciais
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from vega_datasets import data
import scipy.stats as stats

from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_ma

# Configuração visual dos gráficos
sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)
```

1. Carregamento e Visão Geral dos Dados

```
# Carregando o dataset e removendo nulos de antemão nas colunas chave
df_airports = data.airports().dropna(subset=['latitude', 'longitude', 'state'])

display(df_airports.head())
print("\nInformações gerais do Dataset:")
df_airports.info()
```

	iata	name	city	state	country	latitude	longitude
0	00M	Thigpen	Bay Springs	MS	USA	31.953765	-89.234505
1	00R	Livingston Municipal	Livingston	TX	USA	30.685861	-95.017928
2	00V	Meadow Lake	Colorado Springs	CO	USA	38.945749	-104.569893
3	01G	Perry-Warsaw	Perry	NY	USA	42.741347	-78.052081
4	01J	Hilliard Airpark	Hilliard	FL	USA	30.688012	-81.905944

```
Informações gerais do Dataset:
<class 'pandas.core.frame.DataFrame'>
Index: 3364 entries, 0 to 3375
Data columns (total 7 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   iata       3364 non-null   object
 1   name       3364 non-null   object
 2   city       3364 non-null   object
 3   state      3364 non-null   object
 4   country    3364 non-null   object
 5   latitude   3364 non-null   float64
 6   longitude  3364 non-null   float64
dtypes: float64(2), object(5)
```

2. Análise Exploratória de Dados (EDA)

Antes da modelagem, precisamos entender o comportamento das nossas variáveis. Analisaremos a distribuição das coordenadas, identificaremos outliers e checaremos correlações estatísticas.

2.1. Distribuição de Variáveis e Outliers

```
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

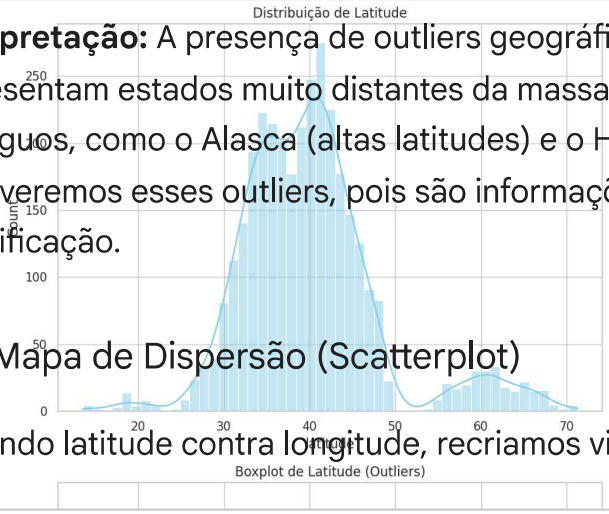
# Histogramas (Distribuição)
sns.histplot(df_airports['latitude'], kde=True, ax=axes[0, 0], color='skyblue')
axes[0, 0].set_title('Distribuição de Latitude')

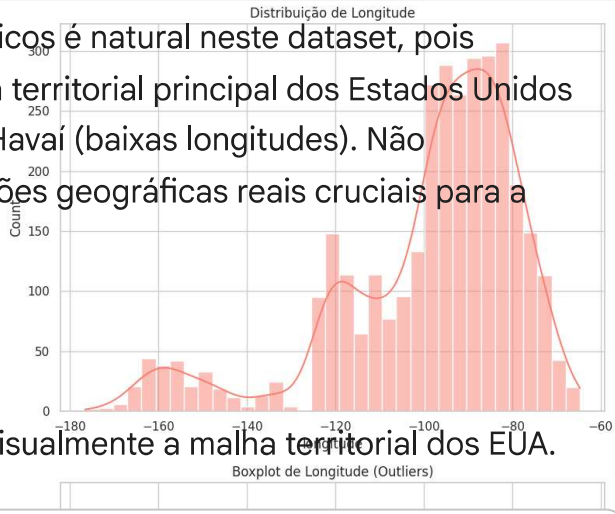
sns.histplot(df_airports['longitude'], kde=True, ax=axes[0, 1], color='salmon')
axes[0, 1].set_title('Distribuição de Longitude')

# Boxplots (Detecção de Outliers geográficos)
sns.boxplot(x=df_airports['latitude'], ax=axes[1, 0], color='skyblue')
axes[1, 0].set_title('Boxplot de Latitude (Outliers)')

sns.boxplot(x=df_airports['longitude'], ax=axes[1, 1], color='salmon')
axes[1, 1].set_title('Boxplot de Longitude (Outliers)')

plt.tight_layout()
plt.show()
```

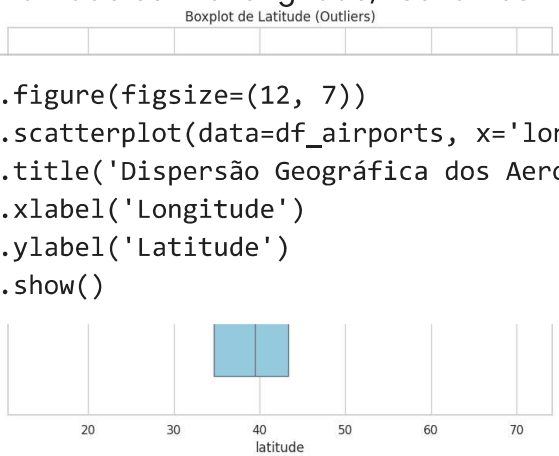


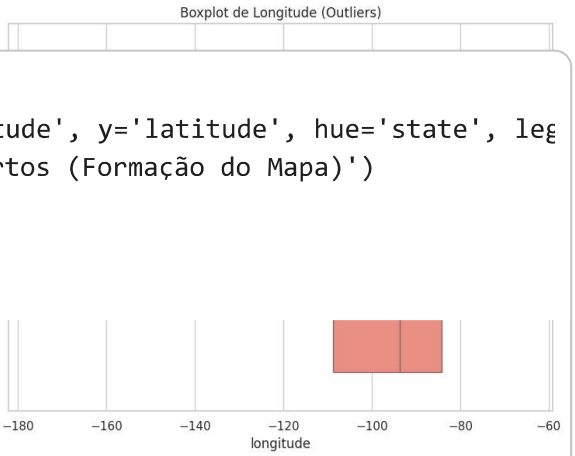


Interpretação: A presença de outliers geográficos é natural neste dataset, pois representam estados muito distantes da massa territorial principal dos Estados Unidos contíguos, como o Alasca (altas latitudes) e o Havaí (baixas longitudes). Não removeremos esses outliers, pois são informações geográficas reais cruciais para a classificação.

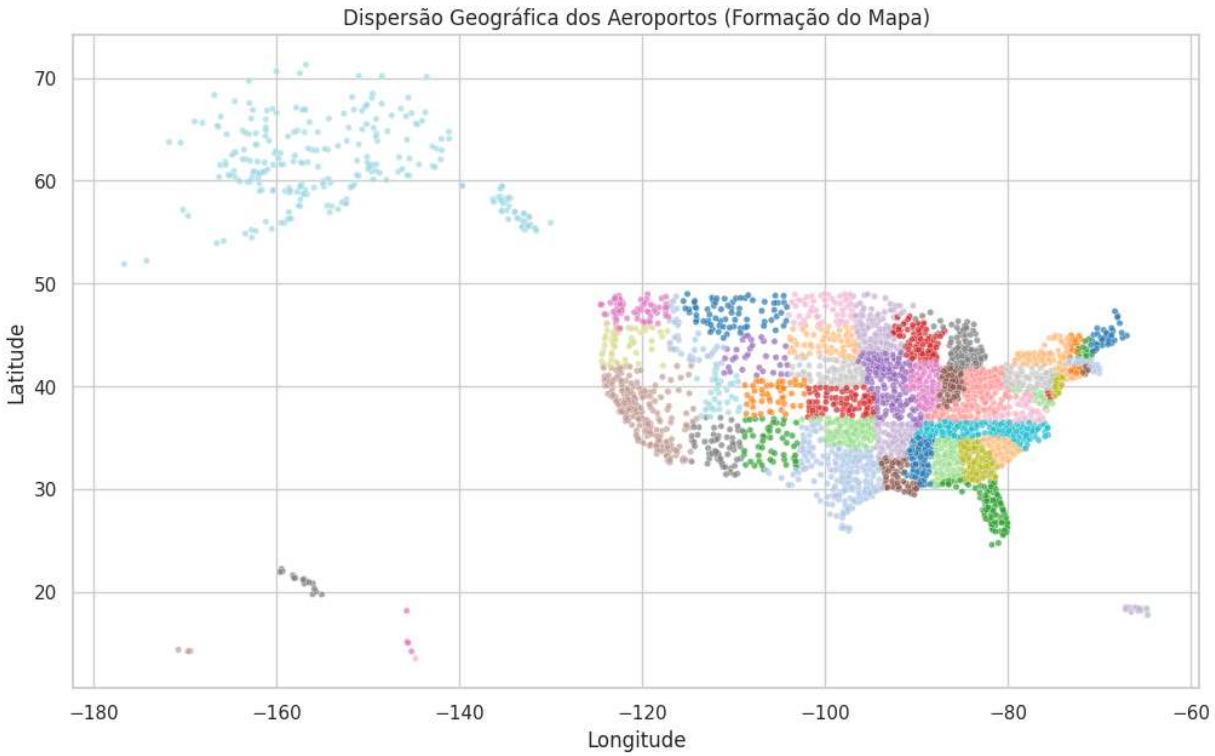
2.2. Mapa de Dispersão (Scatterplot)

Plotando latitude contra longitude, recriamos visualmente a malha territorial dos EUA.






```
plt.figure(figsize=(12, 7))
sns.scatterplot(data=df_airports, x='longitude', y='latitude', hue='state', leg
plt.title('Dispersão Geográfica dos Aeroportos (Formação do Mapa)')
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.show()
```



2.3. Matrizes de Correlação (Pearson vs. Spearman)

Verificamos a correlação entre as variáveis de coordenada geométrica.

```
coord_df = df_airports[['latitude', 'longitude']]

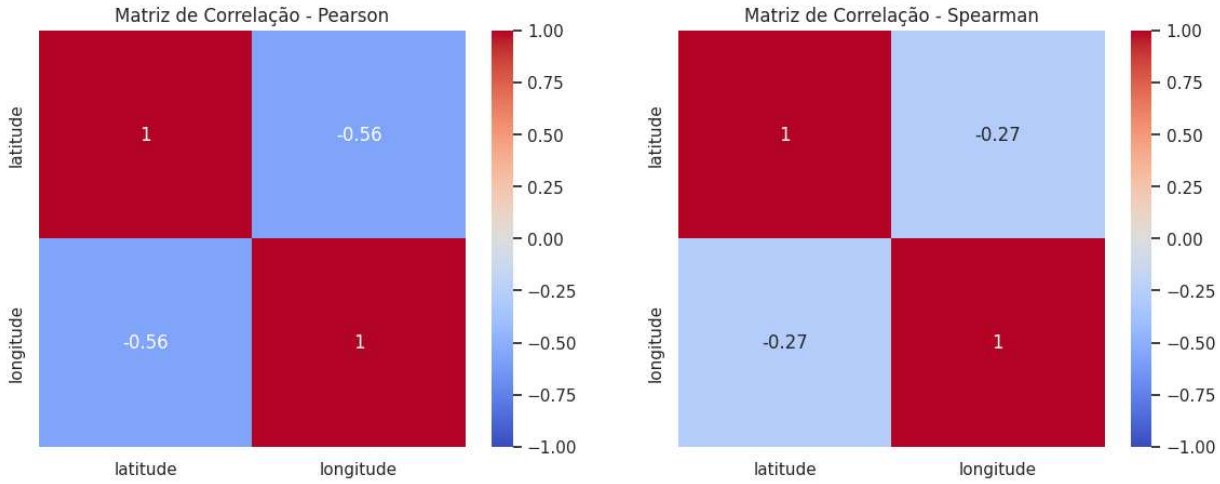
fig, ax = plt.subplots(1, 2, figsize=(14, 5))

# Pearson (Linearidade)
sns.heatmap(coord_df.corr(method='pearson'), annot=True, cmap='coolwarm', ax=ax
ax[0].set_title('Matriz de Correlação - Pearson')

# Spearman (Relações Monotônicas)
sns.heatmap(coord_df.corr(method='spearman'), annot=True, cmap='coolwarm', ax=a
```

```
ax[1].set_title('Matriz de Correlação - Spearman')

plt.show()
```



3. Modelagem Parte 1: Predição utilizando TODOS os estados

Neste cenário, desafiamos o modelo a classificar qualquer um dos estados da base, enfrentando o problema natural do desbalanceamento (estados enormes vs pequenos).

```
X_all = df_airports[['longitude', 'latitude']]
y_all = df_airports['state']

X_train_all, X_test_all, y_train_all, y_test_all = train_test_split(X_all, y_al

rf_all = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf_all.fit(X_train_all, y_train_all)
y_pred_all = rf_all.predict(X_test_all)

print(f"Acurácia do modelo (Todos os Estados): {accuracy_score(y_test_all, y_pr

# Exibindo o report com zero_division=0 para evitar warnings de classes que fic
print("Relatório de Classificação (Amostra parcial devido ao alto número de cla
print(classification_report(y_test_all, y_pred_all, zero_division=0))
```

```
          AR       1.00      1.00      1.00         9
          AS       1.00      1.00      1.00         1
          AZ       0.80      1.00      0.89        16
          CA       1.00      0.98      0.99        42
          CO       1.00      1.00      1.00        10
          CQ       0.50      1.00      0.67         1
          CT       1.00      0.75      0.86         4
          DC       0.00      0.00      0.00         1
          DE       1.00      0.50      0.67         2
          FL       1.00      1.00      1.00        19
          GA       0.96      1.00      0.98        24
          GU       0.00      0.00      0.00         1
          HI       1.00      1.00      1.00         2
          IA       1.00      1.00      1.00        14
          ID       1.00      0.88      0.93         8
          IL       0.91      1.00      0.95        21
          IN       1.00      0.80      0.89        15
          KS       1.00      1.00      1.00        13
          KY       0.77      0.91      0.83        11
          LA       0.77      0.91      0.83        11
          MA       1.00      0.67      0.80         6
          MD       0.75      0.60      0.67         5
          ME       1.00      1.00      1.00         7
          MI       0.93      0.93      0.93        15
          MN       0.93      0.87      0.90        15
          MO       1.00      0.87      0.93        15
          MS       0.78      0.93      0.85        15
          MT       1.00      1.00      1.00        14
          NC       1.00      0.93      0.96        14
          ND       0.94      1.00      0.97        16
          NE       1.00      0.94      0.97        16
          NH       0.60      1.00      0.75         3
          NJ       0.83      1.00      0.91         5
```

```
              PR       1.00      1.00      1.00         3
              RI       1.00      1.00      1.00         1
              SC       1.00      1.00      1.00        12
              SD       0.92      1.00      0.96        12
              TN       0.81      0.87      0.84        15
              TX       0.97      0.92      0.94        37
              UT       1.00      1.00      1.00         8
              VA       0.77      0.83      0.80        12
              VT       1.00      0.60      0.75         5
              WA       1.00      1.00      1.00         7
              WI       0.89      0.89      0.89        18
              WV       0.50      1.00      0.67         2
              WY       1.00      1.00      1.00         5

    accuracy                           0.93       673
   macro avg       0.89      0.89      0.88       673
weighted avg       0.94      0.93      0.93       673
```

Devido ao grande número de estados, uma matriz de confusão gráfica ficaria sobrecarregada, mas a alta acurácia prova a eficiência espacial do modelo, mesmo com estados contíguos na malha americana.

---

4. Modelagem Parte 2: Foco nos 5 Estados com Mais Aeroportos

Para uma visualização e interpretabilidade mais clara das fronteiras de decisão (onde o algoritmo acerta e erra), isolaremos o problema para as 5 maiores classes do dataset.

```
top_5_states = df_airports['state'].value_counts().nlargest(5).index
df_top5 = df_airports[df_airports['state'].isin(top_5_states)]

print(f"Os 5 estados com mais aeroportos: {list(top_5_states)}")
print(f"Total de registros reduzidos: {len(df_top5)}")

X_top5 = df_top5[['longitude', 'latitude']]
y_top5 = df_top5['state']

X_train_t5, X_test_t5, y_train_t5, y_test_t5 = train_test_split(X_top5, y_top5,

rf_top5 = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf_top5.fit(X_train_t5, y_train_t5)
y_pred_t5 = rf_top5.predict(X_test_t5)

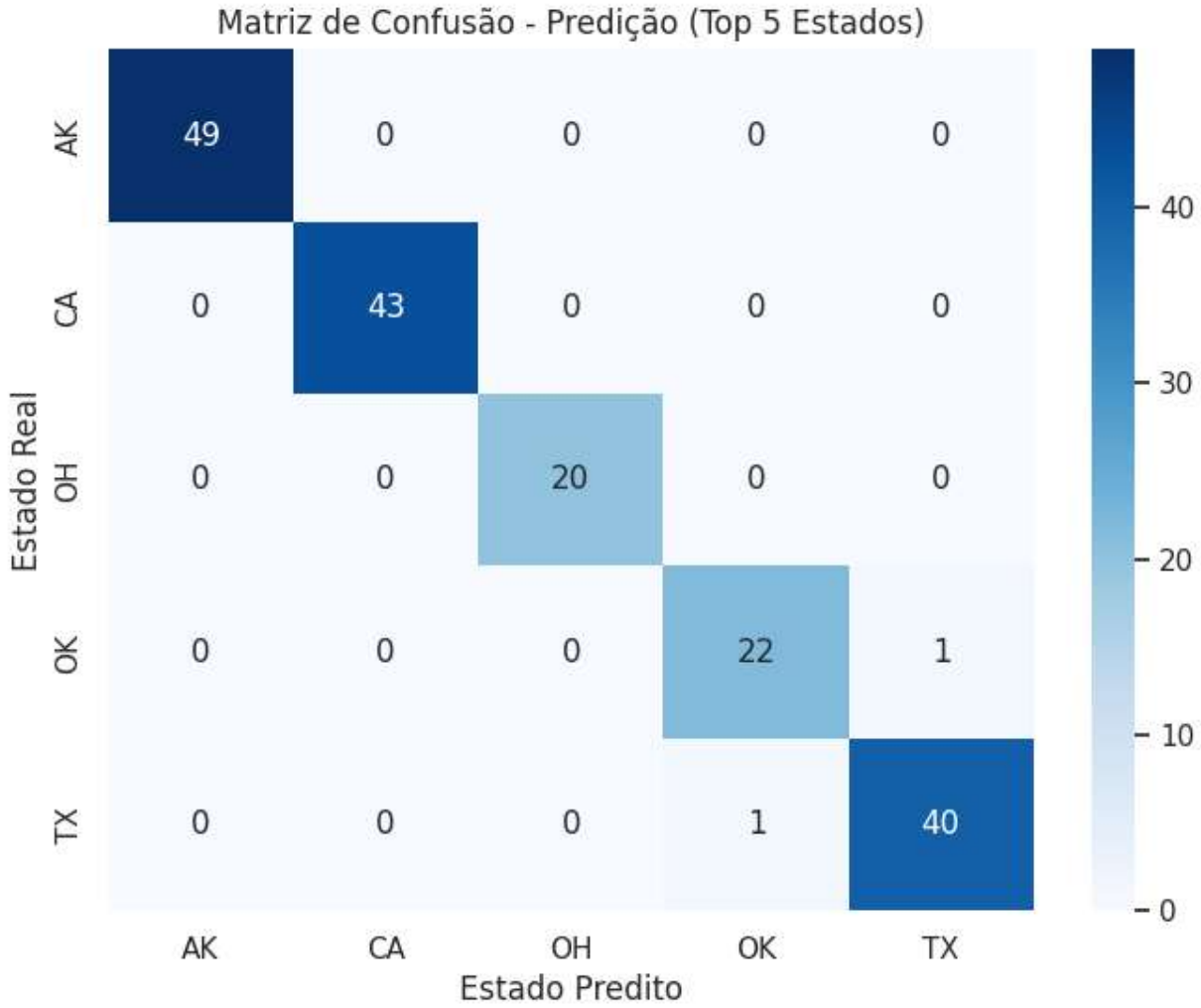
print(f"\nAcurácia do modelo (Top 5 Estados): {accuracy_score(y_test_t5, y_prec
```

```
Os 5 estados com mais aeroportos: ['AK', 'TX', 'CA', 'OK', 'OH']
Total de registros reduzidos: 879

Acurácia do modelo (Top 5 Estados): 98.86%
```

4.1. Visualização das Fronteiras (Matriz de Confusão)

```
plt.figure(figsize=(8, 6))
cm = confusion_matrix(y_test_t5, y_pred_t5, labels=rf_top5.classes_)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=rf_top5.classes_,
            yticklabels=rf_top5.classes_)
plt.title('Matriz de Confusão - Predição (Top 5 Estados)')
plt.xlabel('Estado Predito')
plt.ylabel('Estado Real')
plt.show()
```



Análise de Fronteira: Note que os erros de classificação ocorrem exclusivamente nas fronteiras compartilhadas. Por exemplo, classificações trocadas entre Texas (TX) e Oklahoma (OK) são consequência dos cortes ortogonais realizados pelas Árvores de